

A parallel block cyclic reduction algorithm for the fast solution of elliptic equations *

E. GALLOPOULOS and Y. SAAD

Center for Supercomputing Research and Development, University of Illinois at Urbana Champaign, Urbana, IL 61801, U.S.A.

Received November 1987

Revised March 1988

Abstract. This paper presents an adaptation of the Block Cyclic Reduction (BCR) algorithm for a multi-vector processor. The main bottleneck of BCR lies in the solution of linear systems whose coefficient matrix is the product of tridiagonal matrices. This bottleneck is handled by expressing the rational function corresponding to the inverse of this product as a sum of elementary fractions. As a result the solution of this system leads to parallel solutions of tridiagonal systems. Numerical experiments performed on an Alliant FX/8 are reported.

Keywords. Block cyclic reduction algorithm, multi-vector processor, parallelization using partial fraction expansions, Alliant FX/8, numerical experiments.

1. Introduction

The numerical solution of linear elliptic partial differential equations is a problem of major importance in many fields of science and engineering. Several techniques have developed in the past two decades to solve separable elliptic problems much faster than the traditional iterative methods [1,4,5,14]. The most commonly used of these *Rapid Elliptic Solvers* (RES) are based on either applying the Fast Fourier Transform (FFT) to decouple the block tridiagonal systems into multiple scalar ones, or on using the general *Block Cyclic Reduction* (BCR) as described in [18], or finally, a combination of the above approaches as is the case in the FACR algorithm, first introduced by Hockney. For example, the solution of Poisson's equation on a rectangle discretized with an $n \times n$ grid, entails an asymptotic operation count of $O(n^3 \log n)$ and $O(n^2 \log^2 n)$ for the (iterative) SOR and ADI methods respectively, but only $O(n^2 \log n)$ for the cyclic reduction and FFT-based methods.

With the advent of vector and parallel architectures, researchers have concentrated their efforts on exploiting the computational resources to achieve the best possible performance of these solvers [7,11]. The FFT-based methods are very suitable for such a task because of the amenability of the FFT to parallel computation and because of the immediate decoupling of the equations into multiple scalar tridiagonal systems. Thus, the development of highly efficient

* A preliminary version of this work appeared in reference [3] and was presented at I.C.S. '87, June 8–12, 1987, Athens, Greece. The authors were supported by the National Science Foundation under Grants No. US NSF-MIP-8410110 and US NSF DCR85-09970, the US Department of Energy under Grant No. DOE DE-FG02-85ER25001, by the US Air Force under Contract AFSOR-85-0211, and the IBM donation.

algorithms for the parallel/vector solution of multiple tridiagonal systems as well as for the FFT were instrumental in the acceptance of this approach. On the other hand, and despite some important advantages, the BCR methods did not fare as well and little can be found in the literature concerning the implementation of BCR-based elliptic problem solvers on the new generation supercomputers. The main reason for this is that as the BCR algorithm progresses, it appears that one must solve a steadily decreasing number of tridiagonal systems, the coefficient matrix of each being matrix polynomial. The difficulty in the standard implementation is that one must solve in sequence a tridiagonal system per polynomial factor.

In this paper we describe a new method which is used to overcome this bottleneck and introduce parallelism at each step of the computation. Our approach can be used with success on most of the new supercomputers. An important advantage of the method is that it allows the efficient parallelization/vectorization of NCAR's FISHPACK (see [16]) which is entirely based on BCR for two-dimensional problems. Moreover, the technique can be easily applied to parallelize the Approximate Cyclic Reduction method, recently proposed by Swarztrauber in [15].

The structure of the paper is as follows: Section 2 describes BCR and the difficulties in a parallel implementation, Section 3 and 5 describe the new algorithm, and Section 4 discusses implementation issues. In Section 6 we present some numerical experiments and in Section 7 we propose a few tentative concluding remarks.

2. Background: The main bottleneck in Buneman's algorithm

Consider a block tridiagonal system of the form

$$\begin{bmatrix} A & -I & & & \\ -I & A & -I & & \\ & & \ddots & \ddots & \\ & & & -I & A & -I \\ & & & & -I & A \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} f_1 \\ f_2 \\ \vdots \\ f_n \end{bmatrix} \quad (2.1)$$

where each block is an $m \times m$ tridiagonal matrix and the sub-vectors v_i and f_i are all of size m . Such systems frequently arise when discretizing an elliptic partial differential equation of the form

$$a(x) \frac{\partial^2 u}{\partial x^2} + b(x) \frac{\partial u}{\partial x} + c(x)u + \frac{\partial^2 u}{\partial y^2} = f(x, y)$$

with Dirichlet boundary conditions over, for example, a rectangular region.

For simplicity we assume at first that the block-size n of the system (2.1) is of the form $n = 2^\mu - 1$. For a detailed description of the algorithm in the general case the reader is referred to [18]. At the r th step of BCR we have a system of the form

$$\begin{bmatrix} A^{(r)} & -I & & & \\ -I & A^{(r)} & -I & & \\ & & \ddots & \ddots & \\ & & & -I & A^{(r)} & -I \\ & & & & -I & A^{(r)} \end{bmatrix} \begin{bmatrix} v_h \\ v_{2h} \\ \vdots \\ v_{n,h} \end{bmatrix} = \begin{bmatrix} f_h \\ f_{2h} \\ \vdots \\ f_{n,h} \end{bmatrix} \quad (2.2)$$

where $h = 2^r$, and each component of the right-hand side is put in the form

$$f_{jh} = A^{(r)} p_{jh}^{(r)} + q_{jh}^{(r)}. \quad (2.3)$$

Initially ($r = 0$) the vectors $p_i^{(0)}$ are zero while $q_i^{(0)} = f_j$. Equations whose block index j is odd are eliminated by multiplying each equation numbered $2jh$ by $A^{(r)}$ and adding to it equations $(2j-1)h$ and $(2j+1)h$. This yields a system of half the size involving only equations whose indices are even multiples of h . Specifically, the general equation becomes

$$\begin{aligned} & -v_{(2j-2)h} + \left((A^{(r)})^2 - 2I \right) v_{2jh} - v_{(2j+2)h} \\ & = (A^{(r)})^2 p_{2jh}^{(r)} + A^{(r)} \left(q_{2jh}^{(r)} + p_{(2j-1)h}^{(r)} + p_{(2j+1)h}^{(r)} \right) + q_{(2j-1)h}^{(r)} + q_{(2j+1)h}^{(r)}. \end{aligned}$$

The above equation is then rewritten such that the new right-hand sides will have a form similar to (2.3) in step $r+1$. This is achieved by defining

$$A^{(r+1)} = (A^{(r)})^2 - 2I, \quad (2.4)$$

$$P_{2jh}^{(r+1)} = p_{2jh}^{(r)} + (A^{(r)})^{-1} \left(q_{2jh}^{(r)} + p_{(2j-1)h}^{(r)} + p_{(2j+1)h}^{(r)} \right), \quad (2.5)$$

$$q_{2jh}^{(r+1)} = 2p_{2jh}^{(r+1)} + q_{(2j-1)h}^{(r)} + q_{(2j+1)h}^{(r)}. \quad (2.6)$$

After $\mu - 1$ steps of these transformations system (2.2) reduces to a system with a single block which can be solved directly. Then the back-substitution phase will consist of computing v_{jh} , for the odd values of j , using the fact that for even values of j , the v_{jh} have been computed in the previous step. Using (2.3), we see that we must compute, for j odd and $jh \leq n$,

$$v_{jh} = p_{jh}^{(r)} + (A^{(r)})^{-1} \left(q_{jh}^{(r)} + v_{(j-1)h} + v_{(j+1)h} \right). \quad (2.7)$$

The above backward substitution steps are performed for $h = 2^r$ decreasing from 2^μ to $2^0 = 1$. In summary, Buneman's variant of BCR takes the following form:

Algorithm 2.1.

Step 1. *Initialize:* $p_i^{(0)} = 0$, $q_j^{(0)} = f_j$, $j = 1, \dots, n$ and $h = 1$, $r = 0$.

Step 2. *Forward solution:*

- Form the matrix Y_r with columns $q_{2jh}^{(r)} + p_{(2j-1)h}^{(r)} + p_{(2j+1)h}^{(r)}$, $j = 1, \dots, n/2h$,
- Solve the (multi)-linear system $A^{(r)} X_r = Y_r$,
- Update the vectors p and q according to (2.5) and (2.6),
- If $h < n$, then $h = 2h$, $r = r + 1$; go to a.

Step 3. *Solve* for u , $A^{(r)} u = q_1^{(r)}$ and set $v_h = p_h + u$.

Step 4. *Backward substitution:* while $h > 1$ do

- $h = h/2$, $r = r - 1$,
- Form the matrix Y_r with column vectors $q_{jh}^{(r)} + v_{(j-1)h} + v_{(j+1)h}$, $j = 1, 3, 5, \dots, n/h$,
- Solve for U_r , $A^{(r)} U_r = Y_r$,
- Update the solution vectors v_{jh} , $j = 1, 3, \dots$ to $jh \leq n$, according to (2.7)

The most time-consuming part of the above algorithm lies in the solution of the linear system in the forward and backward phases, i.e., in Step 2(b) and Step 4(c). Clearly, the matrix $A^{(r)}$ is never formed explicitly. In fact the way in which Step 2(a) and Step 4(c) are usually performed is by first observing that the matrix $A^{(r)}$ is a known polynomial in A , specifically [18]

$$A^{(r)} = 2T_{2^r} \left(\frac{A}{2} \right) \equiv \prod_{i=1}^{2^r} (A - \lambda_i I)$$

where T_k denotes the Chebyshev polynomial of the first kind of degree k and therefore

$$\lambda_i = 2 \cos \frac{(2i-1)\pi}{2^{r+1}}, \quad i = 1, 2, \dots, 2^r.$$

When implemented on a serial computer, the linear systems corresponding to each of the factors and each of the right-hand sides are solved in sequence by Gaussian elimination. On vector and parallel machines, this result is an inefficient utilization of the available computational resources. An obvious but partial remedy is to still use Gaussian elimination but solve for all the right-hand sides simultaneously. This will achieve good performance at the beginning of the forward sweep and at the end of the backward sweep since there are many such right-hand sides. However, looking at the forward solution step only, the number of right-hand sides decreases exponentially $((n-1)/2$ then $(n-1)/4, \dots, 1)$, while the number of factors increases in the reverse order. This means that this high performance can only be achieved at the very first few steps. A similar situation occurs in the backward solution phase where the number of right-hand sides starts at one and then doubles at every step to reach the maximum number $(n+1)/2$ at the end of Step 4.

Another way of introducing parallelism is to use cyclic reduction on each tridiagonal system. However, the cost of cyclic reduction in terms of operation count is high and its use may lead to disappointing speedups. This was in particular pointed out in [10] in the context of Alternating Direction Implicit (ADI) methods.

3. The new algorithm¹

In this section, we continue with the assumption that we are solving a problem with Dirichlet conditions on the boundary and with a rectangular grid of dimension $n \times m$ with $n = 2^\mu - 1$. The basic problem to be solved at each step of BCR is of the form

$$\prod_{i=1}^{i=k} (A - \lambda_i I) X = Y \quad (3.1)$$

in which X and Y are matrices of dimension $m \times \nu$. The usual way of solving (3.1) is by means of the following algorithm:

Algorithm 3.1.

Step 1. Set $X_0 = Y$.

Step 2. Do $i = 1, \dots, k$

a. Set $Y_i = X_{i-1}$,

b. Solve $(A - \lambda_i I) X_i = Y_i$.

Step 3. $X = X_k$

As was pointed out earlier when ν is large enough, the amount of vectorization and parallelization that is available in each of the inner loops of the above algorithm with the standard Gaussian elimination may be sufficient to deliver reasonable performance. The vector length with this simple approach is ν so that when ν is small, the delivered speed may become inadequate.

To introduce the alternative we propose, we start by observing that we are seeking the matrix $[g_k(A)]^{-1}Y \equiv s_k(A)Y$ where s_k is the rational function

$$s_k(t) \equiv \frac{1}{g_k(t)} \equiv \frac{1}{\prod_{i=1}^k (t - \lambda_i)}. \quad (3.2)$$

We will need the following well-known elementary result:

¹ It was discovered recently by the authors that the idea of using partial fraction expansions in the context of BCR was presented by R. Sweet at the *International Conference on Vector and Parallel Computing*, held in Loen, Norway, June 2-6, 1986. A written exposition of these results can be found in [18].

Lemma 3.2. *Every rational function*

$$f(t) = \frac{u_l(t)}{g_k(t)} = \frac{u_l(t)}{\prod_{i=1}^k (t - \lambda_i)},$$

where the degree l of u_l is less than k and where the poles λ_i are all distinct, can be expanded in terms of elementary fractions as follows:

$$f(t) = \sum_{i=1}^k \frac{\alpha_i}{t - \lambda_i}$$

where the scalars α_i are given by

$$\alpha_i = u_l(\lambda_i) / g'_k(\lambda_i).$$

In the context of BCR the polynomials involved are Chebyshev polynomials of the first and second kind, so that the partial fraction coefficients can be easily calculated (cf. Section 5). For example, in the case we are describing, f is given by (3.2) with $g_k(t) = 2T_k(\frac{1}{2}t)$ and a little calculation shows that

$$\alpha_i = \frac{(-1)^{i-1}}{2^r} \cdot \sin \frac{(2i-1)\pi}{2^{r+1}}.$$

As a result we can decompose the solution X as

$$X = \sum_{i=1}^k \alpha_i (A - \lambda_i I)^{-1} Y.$$

This leads to the following rival for Algorithm 3.1:

Algorithm 3.3.

Step 1. Do for $i = 1, 2, \dots, k$

Solve $(A - \lambda_i I)X_i = Y$.

Step 2. Compute $X = \sum_{i=1}^k \alpha_i X_i$.

The clear advantage of Algorithm 3.3 over Algorithm 3.1 is that the tridiagonal systems involving the matrix $(A - \lambda_i I)$ are now decoupled and can be solved in parallel. In essence we have replaced a problem involving sequential solutions of tridiagonal linear systems by one which involves the simultaneous solution of independent tridiagonal linear systems and the linear combination of the partial results.

Two important questions might be asked at this point. First there is the danger of instability: the algorithm is not acceptable if it produces wrong answers. A simple analysis based on the exact expressions of the α_i 's shows that the process is stable. This fact is also confirmed by the experiments. The second question concerns cost. The summation in Step 2 of the algorithm involves an additional $2k \times m \times v \approx m \times n$ operations to be added to the usual $5 \times m \times n + 5 \times k \times m$ associated with the traditional Algorithm 3.1. Thus the additional operation count is not negligible: approximately 20% for small k and 10% for large k . The above are approximate operation counts obtained by dropping the less significant terms. Note, however, that this additional work is perfectly amenable to parallelization and high performance can be achieved.

4. Implementation issues

The most difficult issue that one faces when implementing the approach outlined in Section 3 is the fact that in the course of BCR, the two parameters k , the degree of the polynomial, and ν the number of right-hand sides, vary widely from one outer step to the other. In fact the relation $k \times (\nu + 1) = \lfloor \frac{1}{2}(n + 1)/2 \rfloor$ holds for the case when n is of the form $2^\mu - 1$ which means that the two numbers k and $\nu + 1$ will change exponentially and in inverse proportion of each other. As a result one must employ different strategies for the various cases: k large but ν small, ν large and k small, etc.

First we discuss one way of implementing the algorithm for a multi-vector processor architecture. The model architecture is that of an Alliant FX/8 which consists of 8 Computational Elements (CEs) each being a vector processor. From the above discussion we must distinguish two cases.

Case 1: ν is large. Here one can vectorize the operations with respect to the ν right-hand sides. Moreover, if k is large enough (larger than the number of CEs), it is important to run concurrently the outer loop of Step 1 in Algorithm 3.3. However, this becomes ineffective as k goes below the number of CEs since then fewer CEs would be active. In this particular case it is preferable to run the outer loop sequentially and perform the vector operations within the subroutine in a vector concurrent mode.

Case 2: k is large. The situation is the opposite of the previous one: vectorization of the operations should now be performed along the variables corresponding to the outer loop of Algorithm 3.3. Again the inner loop of Step 1 in Algorithm 3.3 should be performed concurrently unless ν the number of right-hand sides is smaller than the number of CEs in which case it should be executed sequentially, with the vector loops run in a vector-concurrent mode.

Next we should say a few words on how Algorithm 3.3 can be implemented on a vector computer. Neither of the two approaches outlined above for the two different cases is likely to be effective because the vector lengths will decrease very quickly. An approach that seems best suited for a vector machine is to exploit the observation that at every step we have in fact a large number, namely $k \times \nu$, of tridiagonal systems to solve simultaneously. This leads to a process which is vectorizable with vector length equal to

$$k\nu = \lfloor \frac{1}{2}(n + 1)/2 \rfloor - k \geq \lfloor \frac{1}{4}(n + 1)/4 \rfloor$$

which will be satisfactory for reasonable size problems. These $k\nu$ simultaneous tridiagonal systems are obtained by simply reproducing the tridiagonal matrix $(A - \lambda_i I)$ involved in the (multi) linear system $(A - \lambda_i I)x = b_i$, $i = 1, 2, \dots, \nu$ for each different right-hand side. Step 1 of Algorithm 3.3 is therefore expanded into the simultaneous solution of exactly $k \times \nu$ independent tridiagonal systems each with a single right-hand side. Although good performance can be achieved when n is large, this approach has a drawback which needs to be evaluated more carefully. Indeed, in order to achieve computations with vector lengths of $k\nu$, we must actually reproduce the computed data corresponding to forward solution in the tridiagonal solve ν times. For example, in the first step we can compute in vector mode all the divisions $1/(a_{11} - \lambda_i)$, $i = 1, 2, \dots, k$, then create a copy of the result for each of the ν right-hand sides. Although this involves no arithmetic, the price to pay may be far from negligible.

5. The general case

Sweet generalized the Buneman variant of BCR for a general number of blocks n and showed how it can be applied for any boundary conditions [18]. We describe here the

coefficients of the expansions necessary for the application of our method for general n as well as Dirichlet or Neumann boundary conditions. As before, the key to the success of the method is the ability to express the matrix blocks at each step of BCR as a rational or polynomial matrix form. In fact, whereas before all the diagonal blocks in (2.2) involved the matrix polynomial $A^{(r)}$, in the general case the last diagonal block in (2.2) will be of the form $B^{(r)-1}C^{(r)}$ which can be shown to be a rational function in A . In particular $B^{(r)}$ and $C^{(r)}$ are matrix polynomials in A of degree k_r and l_r , respectively, with $k_r = l_r + 2'$. Depending on the boundary conditions and dimensions, these will be Chebyshev polynomials of the first or the second kind. Specifically, following the notation in [18], the systems that must be solved at each step of the Buneman variant of BCR have one of the following five forms:

$$2T_{2'}(\tfrac{1}{2}A)X = Y, \quad (5.1)$$

$$U_{k_r}(\tfrac{1}{2}A)X = U_{l_r}(\tfrac{1}{2}A)Y, \quad (5.2)$$

$$U_{k_r}(\tfrac{1}{2}A)X = 2U_{l_r}(\tfrac{1}{2}A)T_{2'}(\tfrac{1}{2}A)Y, \quad (5.3)$$

$$T_{k_r}(\tfrac{1}{2}A)X = T_{l_r}(\tfrac{1}{2}A)Y \quad (5.4)$$

and

$$T_{k_r}(\tfrac{1}{2}A)X = 2T_{l_r}(\tfrac{1}{2}A)T_{2'}(\tfrac{1}{2}A)Y \quad (5.5)$$

where T_k and U_k denote, respectively, the Chebyshev polynomials of the first kind

$$T_k(\cos \theta) = \cos(k\theta),$$

and of the second kind

$$U_k(\cos \theta) = \frac{\sin((k+1)\theta)}{\sin \theta}.$$

Note in particular that (5.1) arises when $n = 2^\mu - 1$ described in the previous section, (5.2) and (5.3) when $n \neq 2^\mu - 1$ and with Dirichlet boundary conditions. (5.1), (5.4) and (5.5) arise in the corresponding circumstances but for Neumann boundary conditions.

In product representation

$$T_k(t) = \frac{1}{2} \prod_{i=1}^k (2t - \rho_k(i)) \quad \text{and} \quad U_k(t) = \prod_{i=1}^k (2t - \sigma_k(i))$$

where

$$\rho_k(i) = 2 \cos \frac{(2i-1)\pi}{2k} \quad \text{and} \quad \sigma_k(i) = 2 \cos \frac{i\pi}{k+1}.$$

From Lemma 3.2 it follows that we need to calculate

(i) for (5.1),

$$\alpha_i = \left[\frac{d}{dt} (2T_{2'}(\tfrac{1}{2}t)) \right]^{-1} \quad \text{at } t = \rho_{2'}(i) \quad (i = 1, \dots, 2');$$

(ii) for (5.2),

$$\beta_i = U_{l_r}(\tfrac{1}{2}t) \left[\frac{d}{dt} U_{k_r}(\tfrac{1}{2}t) \right]^{-1} \quad \text{at } t = \sigma_{k_r}(i) \quad (i = 1, \dots, k_r);$$

(iii) for (5.3),

$$\gamma_i = 2U_{l_r}(\tfrac{1}{2}t)T_{2'}(\tfrac{1}{2}t) \left[\frac{d}{dt} (U_{k_r}(\tfrac{1}{2}t)) \right]^{-1} \quad \text{at } t = \sigma_{k_r}(i) \quad (i = 1, \dots, k_r);$$

(iv) for (5.4),

$$\delta_i = T_{l_r}(\tfrac{1}{2}t) \left[\frac{d}{dt} T_{k_r}(\tfrac{1}{2}t) \right]^{-1} \quad \text{at } t = \rho_{k_r}(i) \quad (i = 1, \dots, k_r);$$

(v) for (5.5),

$$\zeta_i = 2T_{l_r}(\tfrac{1}{2}t)T_{2r}(\tfrac{1}{2}t) \left[\frac{d}{dt} (T_{k_r}(\tfrac{1}{2}t)) \right]^{-1} \quad \text{at } t = \rho_{2r}(i) \quad (i = 1, \dots, k_r).$$

We note that in some cases, the degrees of the matrix polynomials in the numerator and denominator of the rational forms may be equal, in which case a simple modification is required in the application of Lemma 3.2. The next lemma follows after some algebraic manipulations and by using the relation $k_r = 2^r + l_r$.

Lemma 5.1.

$$\alpha_i = \frac{(-1)^{i-1}}{2^r} \sin \frac{(2i-1)\pi}{2^{r+1}},$$

$$\beta_i = \frac{2}{k_r+1} \sin \frac{2^r i \pi}{k_r+1} \sin \frac{i \pi}{k_r+1},$$

$$\gamma_i = \frac{2}{k_r+1} \sin \frac{2^{r+1} i \pi}{k_r+1} \sin \frac{i \pi}{k_r+1},$$

$$\delta_i = \frac{2}{k_r} \sin \frac{2^{r-1}(2i-1)\pi}{k_r} \sin \frac{(2i-1)\pi}{2k_r}$$

and

$$\zeta_i = \frac{2}{k_r} \sin \frac{2^r(2i-1)\pi}{k_r} \sin \frac{(2i-1)\pi}{2k_r}.$$

6. Numerical experiments

We have tested our method on Poisson's equation

$$u_{xx} + u_{yy} = f(x, y), \tag{6.1}$$

on the unit square with Dirichlet boundary conditions. The right-hand side f and the Dirichlet boundary conditions are defined so that the true solution is

$$u(x, y) = \sin \left[\frac{\pi(x-y+2)^5}{1+(x-y+2)^2} \right].$$

This is identical with problem 11 of [8].

All of the experiments were performed in IEEE double precision arithmetic on the CSRD Alliant FX/8 system running version 3 of the Concentrix operating system.

6.1. Square grids

In the first experiment, the resolution was chosen so that the number of points in each direction is n . The resulting matrix has block dimension $n \times n$ with each block being of size $n \times n$. Table 1 shows the timings for the original and modified versions of the NCAR package on selected square grids, each but the finest grid run with 1, 2, 4, 6 and then 8 processors.

Table 1
Runtimes of the original and modified FISHPACK on Alliant FX/8

CEs	Dimension	Original (s)	New (s)	Dimension	Original (s)	New (s)
8	31	0.07	0.04	127	1.43	0.42
6	31	0.07	0.05	127	1.46	0.50
4	31	0.07	0.05	127	1.48	0.56
2	31	0.08	0.07	127	1.63	0.89
1	31	0.09	0.11	127	1.92	1.51
8	63	0.31	0.13	255	6.52	1.37
6	63	0.32	0.16	255	6.60	1.61
4	63	0.33	0.18	255	6.80	1.90
2	63	0.35	0.28	255	7.75	3.07
1	63	0.41	0.47	255	8.85	5.30
8				511	29.70	5.44

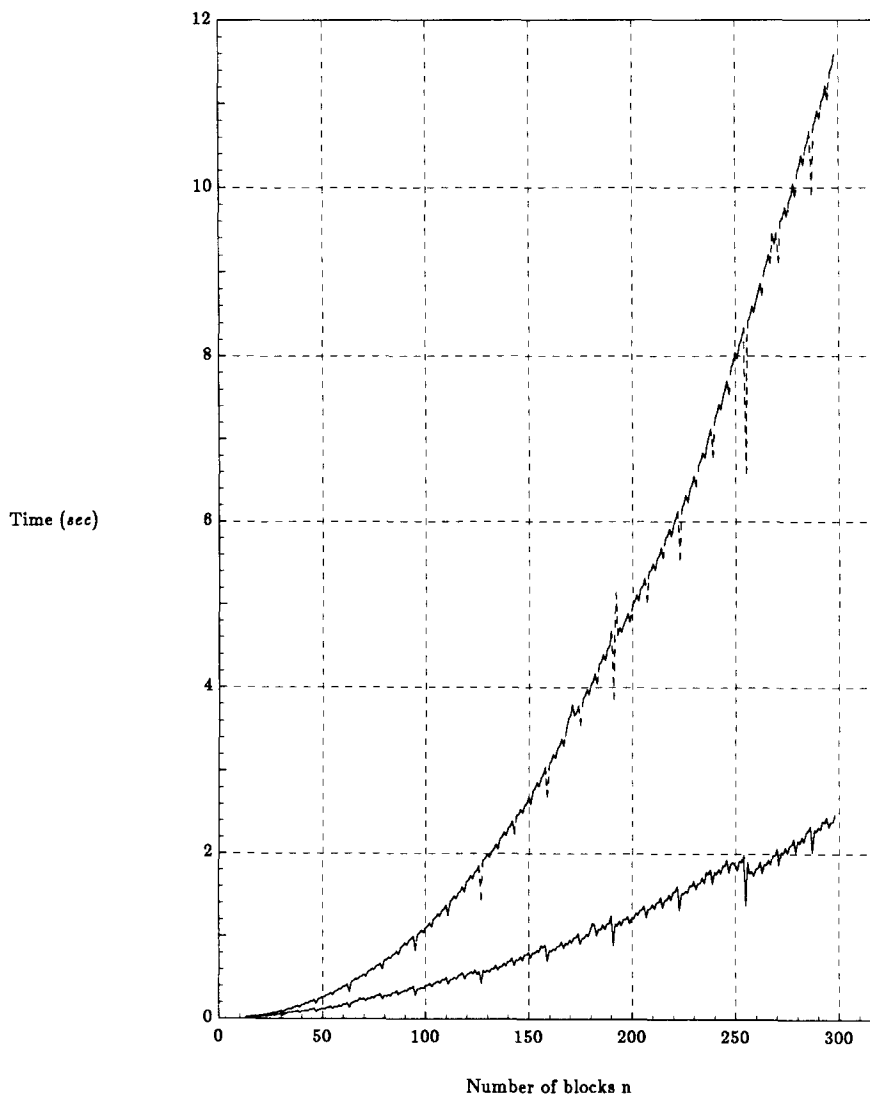


Fig. 1. Old and new methods on $n \times n$ grids.

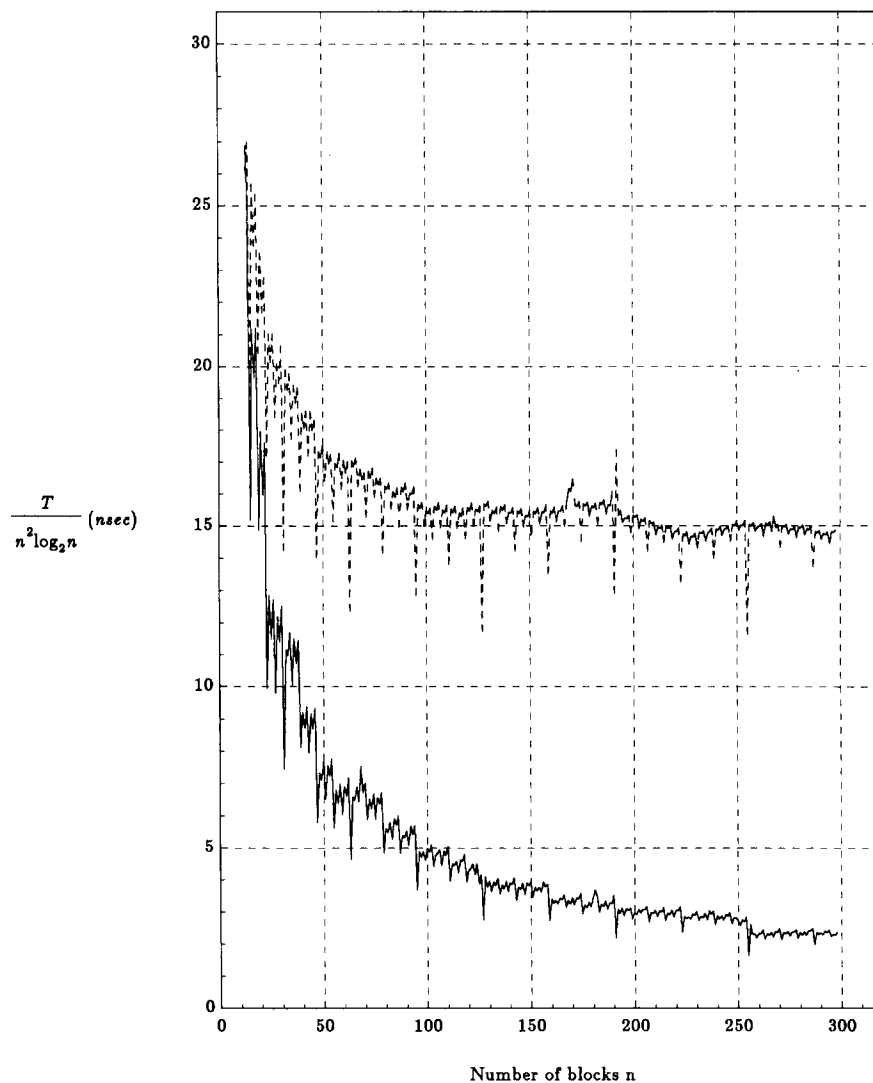
Table 2

Discretization error of the original and modified FISHPACK on Alliant FX/8

Dimension	Original error	New error
31	$1.744869853013731 \times 10^{-2}$	$1.744869853013731 \times 10^{-2}$
63	$4.353361935418354 \times 10^{-3}$	$4.353361935416911 \times 10^{-3}$
127	$1.085979096714262 \times 10^{-3}$	$1.085979096718703 \times 10^{-3}$
255	$2.715698151319890 \times 10^{-4}$	$2.715698151343204 \times 10^{-4}$

Figure 1 depicts the timings for the original and modified versions of FISHPACK running on 8 processors. Observe that as the grid-sizes increase the gains in total time appreciate steadily. This is to be expected from the very sequential nature of the original FISHPACK.

We should point out that the times reported are the total times and do not reflect the gains made in the steps which have been optimized namely the steps corresponding to solving the

Fig. 2. Old and new methods on $n \times n$ grids.

linear systems. If these were to be timed separately, these gains could be substantially better especially for the smaller grid-size problems. For example it is estimated that for the 255×255 grid, the time not attributed to the solution of linear systems takes up to 0.5 s.

For small grids the total gain in the linear systems is somewhat masked by the time spent in the parts of the algorithm that are not related to solving linear systems. Moreover, the benefits of vectorization and parallelization become visible only after a minimal vector length is available. Typically, on the Alliant FX/8, vector processing on each CE becomes more effective than scalar processing after the vector length exceeds 4.

To verify that the method delivers reasonable answers, we compare in Table 2 the infinity norm of the difference between the true solution of (6.1) and the approximate solutions obtained from the original and modified methods. Supporting our previous comments regarding the stability of the new algorithms, we found that the maximum difference between the true and the computed solutions was almost the same for the two methods.

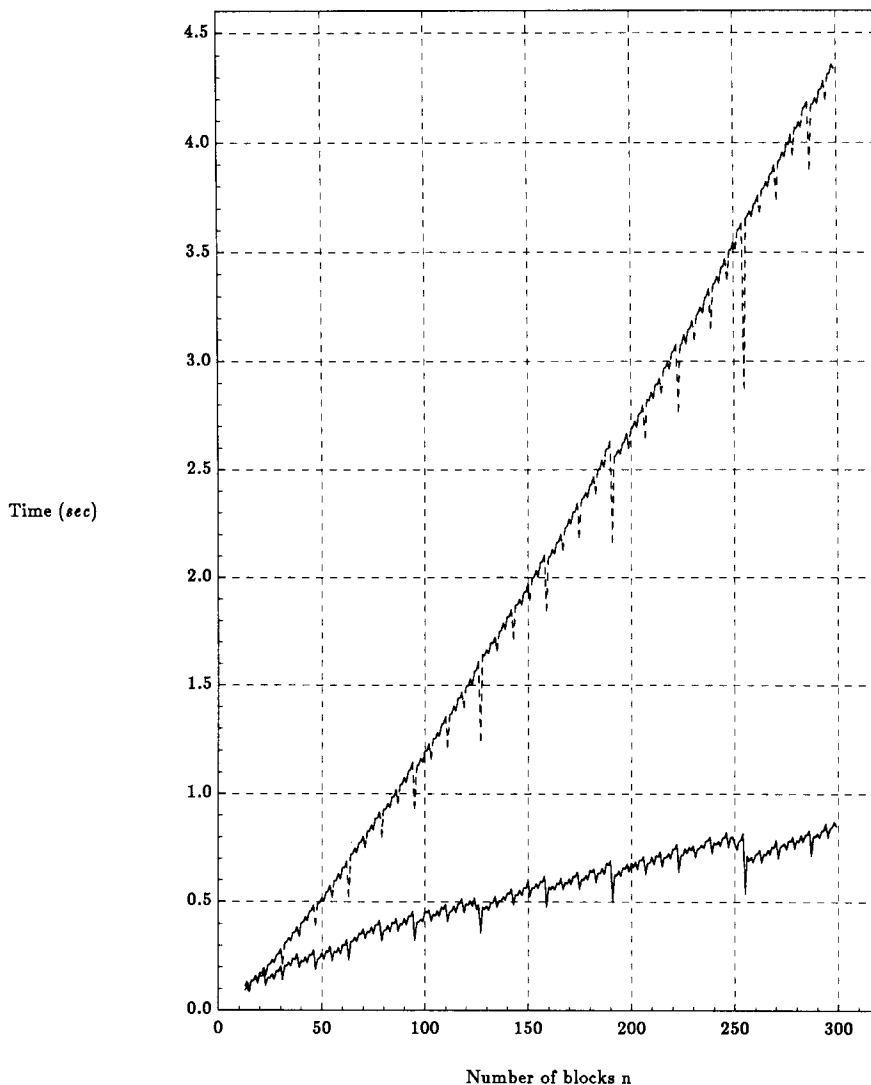


Fig. 3. Old and new methods on $n \times 110$ grids.

In Fig. 2, we show the ratios of the actual times over the leading term $n^2 \log_2 n$ in the computational complexity, for both methods. In a scalar machine, or for a scalar algorithm this function of n should be nearly constant, with some jumps every time n crosses a multiple of a large power of 2. This is more or less verified by the original FISHPACK for larger values of n . On the other hand, it seems to be a steadily decreasing function of n for the new algorithm.

Finally, we observe from Table 1, that for very small grid sizes the new algorithm may be slightly slower than the original one when execution is performed on one CE. Our algorithm has been optimized for the case where the number of CEs is 8 and does not attempt to reduce the overhead when the number of processors is small. Here it would have been better to use the alternative proposed for vector processors as discussed in Section 4.

6.2. Experiments with rectangular grids

In the second experiment, the resolution was chosen to correspond to a $m \times n$ grid, resulting in a matrix of block dimension $n \times n$ and block size $m \times m$.

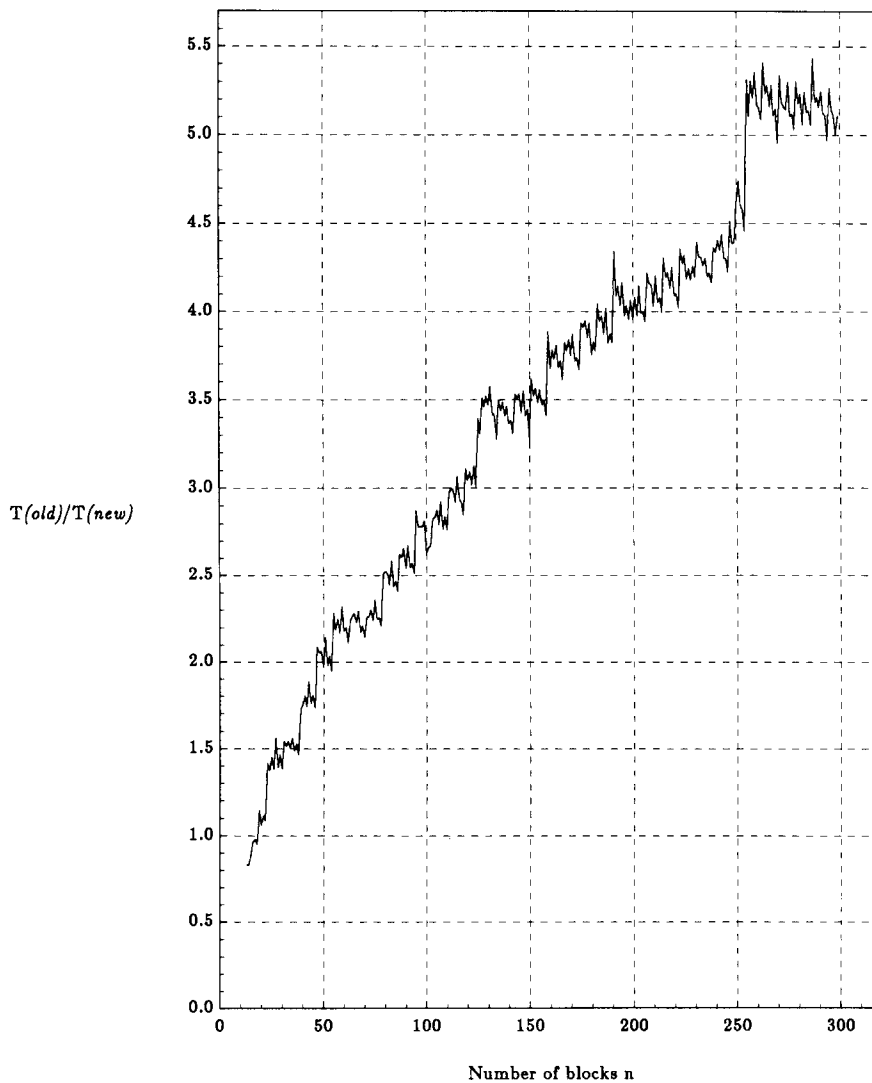


Fig. 4. Old and new methods on $n \times 110$ grids.

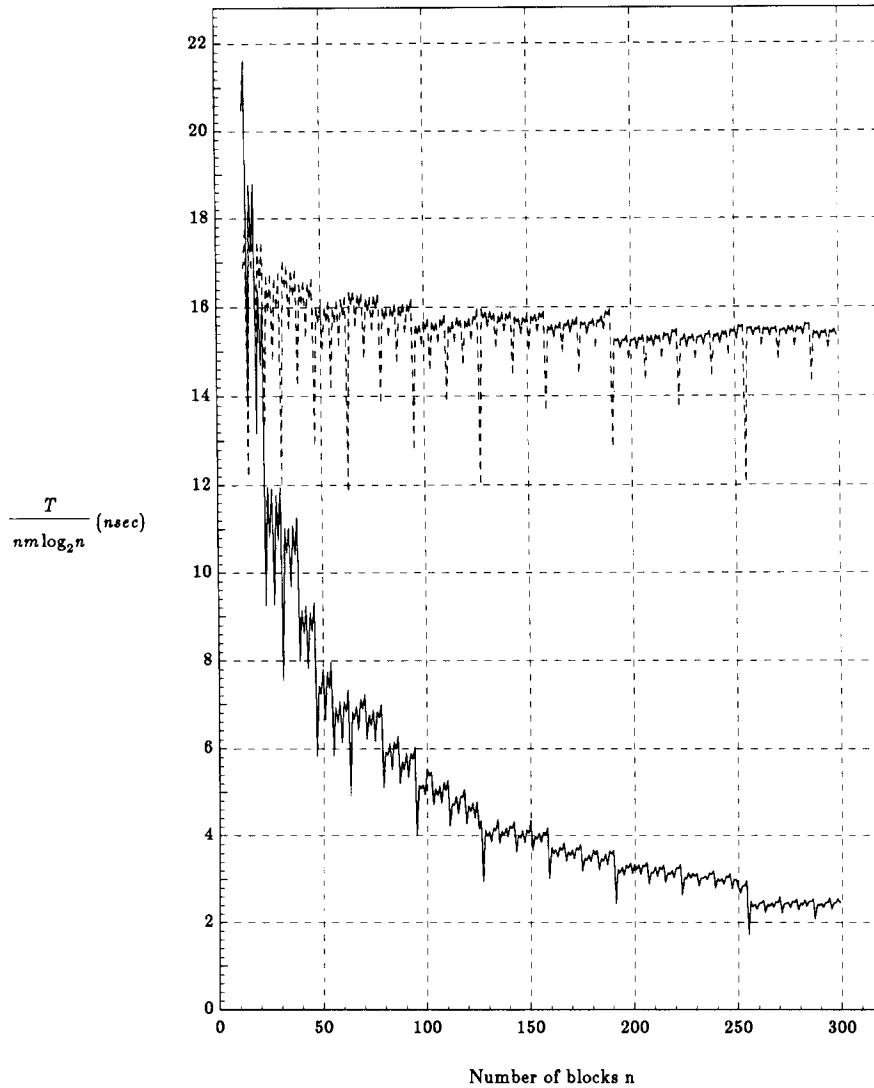


Fig. 5. Old and new methods on $n \times 110$ grids.

Similarly to Fig. 1, we have plotted in Fig. 3 the timings for the original and modified versions of FISHPACK running on 8 processors. Figure 4 depicts the ratios of the original timings versus the new ones for the same problem. An interesting point to be made here is that the time for the original FISHPACK shows a nearly linear behavior with respect to n , instead of superlinear in n . The explanation for this is that the amount of data to be used quickly exceeds the cache size and therefore, the total time is dominated by data movements, which is linear in n .

Similarly to Fig. 2, we also plotted the ratio of the execution time over the leading term $n \times m \times \log_2 n$ in the number of arithmetic operations. This is shown in Fig. 5 and we can make similar comments to those of Section 6.1.

6.3. The effects of parallelism and vectorization

In the following experiments we would like to illustrate the effect of parallelism and vectorization in the new algorithm. The problem considered is the same as before and we first

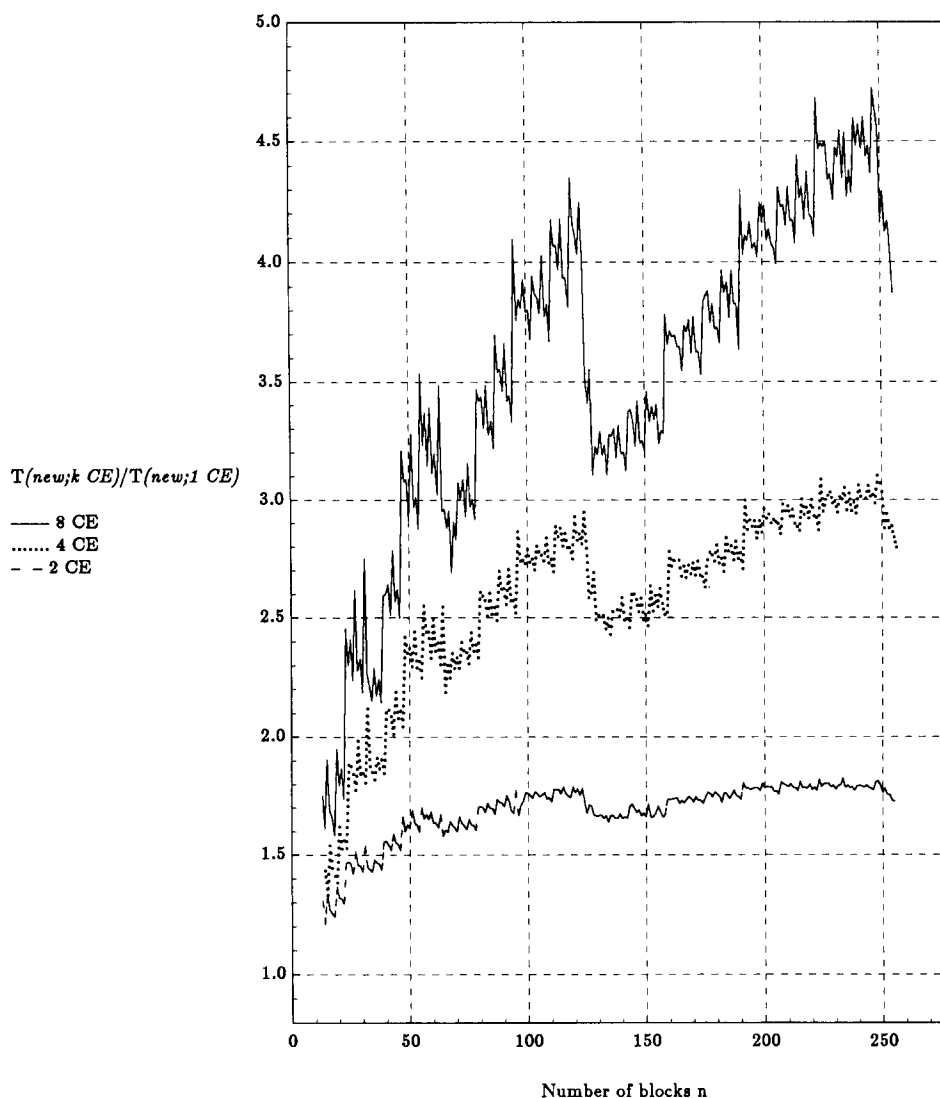
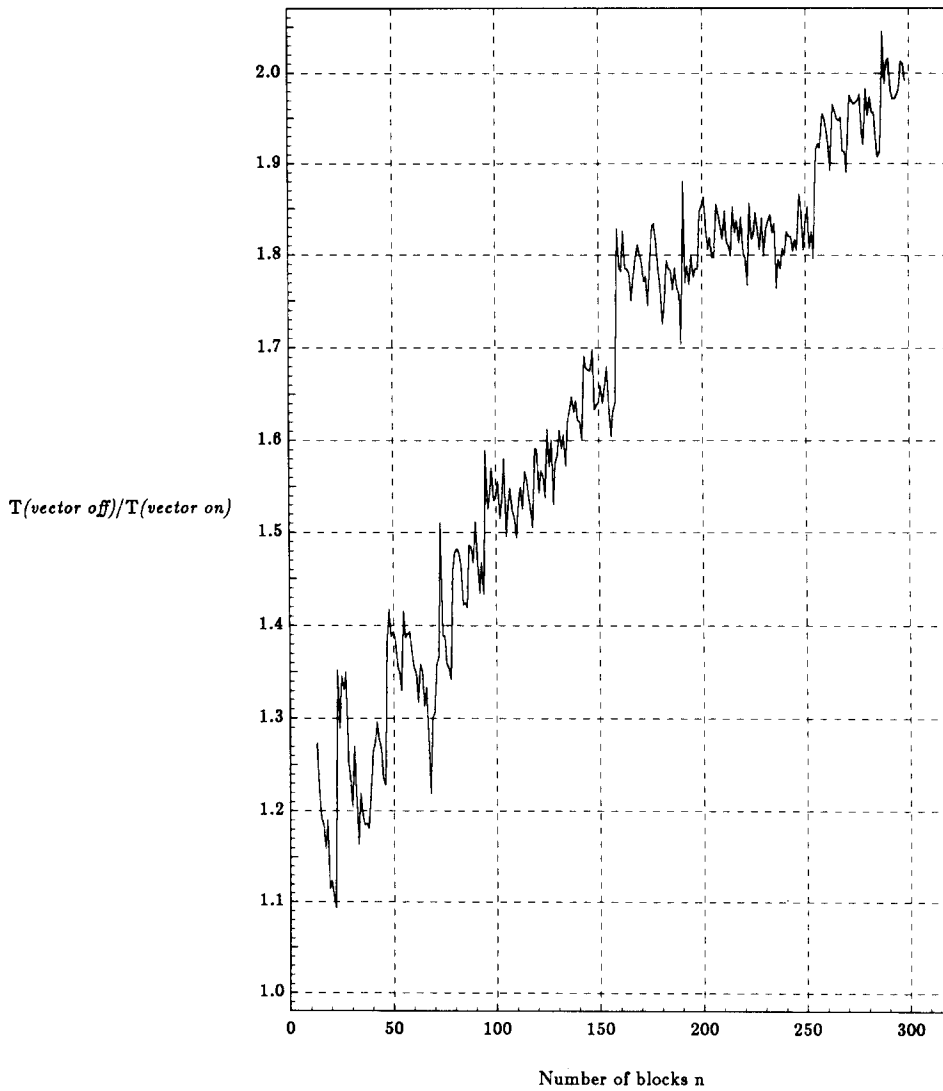


Fig. 6. New method speedup with 2, 4, 8 over 1 CE on $n \times n$ grids.

consider square grids. Figure 6 shows the speedups of the new method when the number of processors is 2, 4, and then 8. The speedup for k CE's is the ratio of the time on one CE over the time on k CEs. As expected, the benefits of parallelism are increased when the grid-size increases. Another interesting observation is that the slopes of the speedup curves tend to be steeper as the number of processors increases. We should point out that we also plotted a similar curve for the original FISHPACK. The results were as follows when the grid size n is between 50 and 250; for 2 CEs the speedup varies around 1.18, for 3 CEs it varies from 1.25 and 1.30 and for 8 CEs it varies between 1.30 and 1.35.

To show the effect of vectorization, we have plotted in Fig. 7, the ratio of the execution times of the new algorithm when vectorization is turned off versus the times of the same algorithm when vectorization is in effect. The best ratio is around 2 and steadily increases with the grid size. It is interesting to point out that for the largest grid size considered in this test ($n = 298$), the speedup over the original package here is around 2.35 versus 4.68 when vectorization is used. One could say that about half of the speedup is to be attributed to

Fig. 7. New method on $n \times n$ grids.

concurrency and the other half to vectorization. However, this is a bit simplistic because it might be possible to gain a better performance by optimizing the code using different strategies for the case where vectorization is off.

7. Other applications

In this section we would like to give an overview of several other potential applications of the use of partial fraction expansions to increase parallelism in numerical algorithms. We start by mentioning that the idea has already been exploited in other fields. For example, [6] uses the same principle in a slightly different form to compute powers of scalars and of matrices. Thus, to compute A^n Kung writes, e.g.,

$$A^n = \left[(A^n - I)^{-1} \right]^{-1} + I = \left[\prod_{i=1}^n (A - \theta_i I)^{-1} \right]^{-1} + I$$

where the θ 's are the n roots of unity. Using the partial fraction expansion to evaluate the term in brackets we notice that A^n can be evaluated with the matrix inversion of the n matrices $(A - \theta_i I)$ in parallel, the linear combination of the results and then another matrix inversion and an addition to the identity matrix. Thus, multiplications have been entirely avoided and replaced by additions and inversions.

Datta and Saad [2] have also used partial fraction expansions in the same way to solve particular Sylvester equations arising in reduced model observers in control. The problem there is to solve a special Sylvester type equation of the form

$$AX - XG = [0, 0, \dots, c]$$

where X is $N \times m$, G is $m \times m$. If G can be chosen to be upper bidiagonal with the super diagonal elements consisting of ones, the i th column x_i of X can be seen to be the solution of $q_{m-i+1}(A)x = c$ where $q_j(t) = (t - \gamma_j) \cdots (t - \gamma_1)$ in which the γ 's are the diagonal entries of G . This can be computed with partial fraction expansions. When G is not of the above bidiagonal form, then an important question is whether or not it is possible to find a solution X that is orthogonal by allowing G to be a Hessenberg matrix whose eigenvalues are preassigned. The answer is yes and the column vectors of the solution X , up to a multiplicative scalar, can be generated by the well-known Arnoldi process started with the special vector

$$x_1 = \rho(A - \lambda_1 I)^{-1}(A - \lambda_2 I)^{-1} \cdots (A - \lambda_m I)^{-1}c$$

where ρ is the normalizing factor. Looking at potential applications in other areas, we believe that there exist many possibilities. One that shows great promise is the problem of computing the product of the exponential of a matrix by a vector, i.e., vectors of the form $w = \exp(A)v$, as occurs in ODE methods. The exponential is typically approximated by a Padé type approximation which leads to a rational function of A applied to a vector. This could be of great importance when devising parallel ODE schemes but also when attempting to solve parabolic type partial differential equations with time-constant coefficients, with the method of lines. Finally, another interesting application is when using shift-and-invert techniques in the context of eigenvalue calculations, in the form suggested by Ruhe [9]. The idea is to project the eigenvalue problem onto a subspace which is spanned by vectors of the form $r(A)v$ where r is a rational function and v is a certain initial vector.

8. Concluding remarks

We have presented and tested an algorithm which is a parallel version of the standard Block Cyclic Reduction algorithm. The numerical results indicate that the method is a promising alternative to FFT-based RES. Moreover, the best potential use of the algorithm is cases where the FFT approach cannot be used. This is because the technique relies on a simple partial fraction expansion. Thus, the same idea can be applied to problems with Neumann or periodic conditions, as well as to problems involving other coordinate systems. Of even more importance is that they can be applied to the more general separable problem

$$a(x) \frac{\partial^2 u}{\partial x^2} + b(x) \frac{\partial u}{\partial x} + c(x)u + d(y) \frac{\partial^2 u}{\partial y^2} + e(y) \frac{\partial u}{\partial y} + g(y)u = f(x, y)$$

originally discussed in [12] in the context of BCR.

In order to fully exploit parallelism and vectorization, great care must be taken in implementing the tridiagonal system solvers. The proper implementation depends on many factors, particularly the architecture available and a good balance between parallelism and

vectorization. Comparisons between BCR, FFT, and FACR remain to be done. We believe that there will be different instances where each of the three approaches is superior.

Finally, as was mentioned in Section 7, there exist several other potential applications of the partial fraction expansions to increase parallelism.

Acknowledgment

We would like to thank K. Gallivan, R. Hockney and R. Sweet for their helpful comments.

References

- [1] B. Buzbee, G. Golub and C. Nielson, On direct methods for solving Poisson's equation, *SIAM J. Numer. Anal.* **7** (1970) 627–656.
- [2] B.N. Datta and Y. Saad, Numerical methods for solving some large scale linear algebra problems in control, Presentation given at the *SIAM Conference on Linear Algebra in Signals, Systems and Control*, 1986, paper in preparation.
- [3] E. Gallopoulos and Y. Saad, A parallel block cyclic reduction algorithm for the fast solution of elliptic equations, CSRD Report 659, University of Illinois, 1987.
- [4] R. Hockney, A fast direct solution of Poisson's equation using Fourier analysis, *J. ACM* **12** (1965) 95–113.
- [5] R. Hockney, The potential calculation and some applications, *Methods Comput. Phys.* **9** (1970) 135–211.
- [6] H.T. Kung, New algorithms and lower bounds for the parallel evaluation of certain rational expressions and recurrences, *J. ACM* **23** (1976) 252–261.
- [7] J. Ortega and R. Voigt, Partial differential equations on vector and parallel computers, *SIAM Rev.* (June 1985) 213–240.
- [8] J. Rice, E. Houstis and W. Dyksen, A population of linear, second order elliptic partial differential equations on rectangular domains. Part 1, *Math. Comp.* **36** (1981) 479–484.
- [9] A. Ruhe, Rational Krylov sequence methods for eigenvalue computations, *Linear Alg. Appl.* (1984) 391–400.
- [10] Y. Saad, On the design of parallel numerical methods in message passing and shared-memory environments, *Proc. International Seminar on Scientific Supercomputer*, Paris (1987).
- [11] A.H. Sameh, S.C. Chen and D.J. Kuck, Parallel Poisson and biharmonic solvers, *Computing* **17** (1976) 219–230.
- [12] P.N. Swarztrauber, A direct method for the discrete solution of separable elliptic equations, *SIAM J. Numer. Anal.* **11** (1974) 1136–1150.
- [13] P.N. Swarztrauber, The methods of cyclic reduction, Fourier analysis and the FACR algorithm for the discrete solution of Poisson's equation on a rectangle, *SIAM Rev.* **19** (1977) 490–501.
- [14] P.N. Swarztrauber, Fast Poisson solvers, in G.H. Golub, ed., *Studies in Numerical Analysis* (Mathematical Association of America, 1984) 319–369.
- [15] P.N. Swarztrauber, Approximate cyclic reduction for solving Poisson's equation, *SIAM J. Sci. Statist. Comput.* **8** (1987) 199–209.
- [16] P. Swarztrauber and R. Sweet, Efficient Fortran subprograms for the solution of elliptic partial differential equations, NCAR Technical Note IA-109, Boulder, (1975).
- [17] R.A. Sweet, A parallel and vector variant of the cyclic reduction algorithm, *SIAM J. Sci. Statist. Comput.*, to appear.
- [18] R.A. Sweet, A cyclic reduction algorithm for solving block tridiagonal systems of arbitrary dimensions, *SIAM J. Numer. Anal.* **14** (1977) 707–720.